MachBoost: Conditional Acceleration for On-Device Text and Visual Models

Vistrit Pandey
vistrit@blinkfire.com

July 2026

Abstract

MachBoost is a local inference package built around a practical observation: repeated work depends on the workload. For text generation, it uses explicit local context to draft token blocks, verifies them with the target model, and returns to native decoding when no candidate exists. For visual question answering, it keeps a model resident and reuses two deterministic products of an unchanged image: projected vision features and the language model's visual-token KV prefix. The two paths share a server and measurement interface but not an acceleration algorithm. We evaluate both on an Apple M1 Max. With `mlx-community/Qwen2.5-3B-Instruct-4bit`, context-backed code continuation reaches a 1.96× median paired speedup over native `mlx-lm`; copied retrieval answers reach 1.58× with experimental one-token re-entry. With `mlx-community/Qwen2.5-VL-3B-Instruct-4bit`, 12 repeated-image questions fall from a 2.861-second uncached median to 0.158 seconds on the combined cache path. The median paired speedup is 17.96×, and median time-to-first-text improves by 19.12×. Every reported visual answer matches its paired uncached answer and the fixture's expected value. This is not a universal 17.96× inference result: the gain removes repeated image encoding and visual-prefix prefill, leaves decode throughput unchanged, and does not apply to a first view or changed image.

1 Introduction

Running LLMs on a laptop has become realistic, but it has not made autoregressive decoding disappear. A small model on Apple Silicon may fit in memory and still feel slow because the loop is serial: produce a token, update the prefix, run the model again. That bottleneck is especially visible in developer workflows, where the model often sits inside an editor, command runner, notebook, or local assistant.

Visual assistants add a different cost. One image may expand into roughly a thousand language-model input positions before the answer begins. Asking a second question about an unchanged screenshot, invoice, diagram, or camera frame can repeat both the vision encoder and the long visual-token prefill. Unlike autoregressive decode, that work is deterministic for a fixed model, processor, and image.

Speculative decoding offers a way around part of this loop. A cheap source guesses several future tokens; the target model checks the guess; accepted tokens are emitted without paying for one full target step each. Prior work usually gets the draft from another model, extra heads, or a reference sequence. MachBoost takes the reference idea and applies it to the local machine. If the user is asking about a repo, config, note, policy, or retrieved passage, the likely continuation may already be present in that local text.

The scope is deliberately modest. MachBoost does not try to accelerate every prompt or modality with one trick. It accelerates grounded text when a verifier can accept useful drafts, and repeated-image queries when the visual input has not changed. If neither condition holds, the request follows the backend's ordinary path.

This paper contributes the following:

- a context-only drafter and initial-candidate gate that avoid treating the prompt wrapper itself as external evidence;
- a cache-aware MLX verifier that fuses continuation and draft scoring, rewinds rejected cache entries in place, and resumes native decoding when useful context ends;
- a paired benchmark against native `mlx-lm` generation with raw rows, run-order alternation, environment provenance, and exact token comparisons;
- a resident local server with warm model lifecycle, streaming Ollama-compatible and OpenAI-compatible endpoints, and raw completion support for editor integrations;
- an MLX-VLM adapter that combines content-addressed projected-feature reuse with image-scoped visual-prefix state, including a Qwen2.5-VL feature-extraction bridge;
- a paired repeated-image benchmark that records cache-level hits, matching prefix length, first-token latency, raw outputs, and expected-answer checks;
- negative experiments with a small draft model, an MTP draft path, DFlash, an Apple Neural Engine draft, and lossless weight compression.

2 Background and Related Work

Speculative decoding was introduced as a way to accelerate autoregressive transformer inference without changing outputs by drafting candidate tokens and verifying them in parallel with the target model [8]. Speculative sampling independently developed a related draft-and-verify method for sampled decoding and reported 2–2.5× speedups in a distributed setting [5]. Subsequent work explored architecture-level or self-drafting variants. Medusa adds multiple decoding heads to predict future tokens and verifies candidate trees [4].

MachBoost is closest to methods that exploit references or prompt overlap rather than an auxiliary draft model. LLMA observes that outputs in retrieval and reference-grounded scenarios often share spans with the input reference, then verifies copied spans to preserve greedy output [16]. Prompt Lookup Decoding replaces the draft model with n-gram matching over the prompt [11]; PLD+ extends that idea by using language-model artifacts such as attention or hidden states to rank candidate spans [12]. Hugging Face Transformers exposes prompt lookup through `prompt_lookup_num_tokens` in assisted generation [7]. MachBoost differs mainly in packaging and operating assumptions: it treats local files and retrieved snippets as ordinary inputs to the accelerator, keeps benchmark data machine-readable, and makes low-overlap prompts a measured fallback case rather than an error.

Results above 2× elsewhere generally require more than process tuning. ReDrafter trains a recurrent draft conditioned on target hidden state and reports up to 2.3× on Apple Silicon [17]. QuantSpec changes the draft's weight and KV-cache representation and reports speedups up to about 2.5× in long-context inference [13]. Mirror-SD maps complementary draft and target work across GPU and NPU devices and reports 2.8–5.8× on server-scale models [3]. At the runtime

layer, BaseRT's native Metal kernels report up to 1.35× over MLX decode rather than a universal 2× [10]. These are different operating points: trained model changes, heterogeneous execution, or low-level kernels. MachBoost's current context path needs none of them, but only applies when the requested continuation overlaps local text.

The same distinction appears outside text generation. Diffusion workloads can reuse intermediate transformations across denoising steps; EasyCache reports 2.1–3.3× for several video models [18]. That mechanism does not transfer directly to autoregressive text decoding. It does, however, support a broader product design in which one package selects a backend-specific accelerator rather than pretending that a single algorithm fits text, image, audio, and video models.

Attention-state reuse is also established. Prompt Cache stores reusable attention states for prompt modules and reports large first-token latency reductions on long repeated prefixes [6]. MLX-VLM provides a prompt-state interface, automatic prefix caching, and an LRU for projected vision features [9]. MachBoost builds on those mechanisms. Its visual adapter uses image content rather than a path string as the cache identity, isolates prompt state by image, and directly invokes the Qwen vision tower when the generic `encode_image` hook is absent. The experiment in this paper measures that composition; it is not evidence that feature caching or prefix caching was invented here.

The implementation targets common local inference stacks. Hugging Face Transformers provides the broad model API surface for Python inference [14]. MLX is an Apple Silicon-oriented array framework and runtime used here for Mac-native evaluation [1]. Text measurements use a Qwen-family model converted for MLX; Qwen3 is the relevant public model-family report [15]. The visual experiment uses Qwen2.5-VL, whose dynamic-resolution vision encoder and document-understanding capabilities are described by Bai et al. [2].

3 Problem Setting

Let M be a target autoregressive model and let x be the prompt token sequence. Greedy decoding emits

$$y_t = \arg\max_v M(v \mid x, y_{<t})$$

for each generation step t. MachBoost receives a corpus C derived from prompt text, retrieved context, local files, or user-provided strings. The goal is to reduce wall-clock latency while returning the same sequence y that greedy decoding with M would return.

This paper stays with greedy decoding. Sampling-compatible speculative decoding is possible in principle, but greedy argmax verification gives a simple correctness check: the accelerated sequence must match the serial baseline token-for-token.

The visual setting is narrower. Let I be an image and q_i a sequence of questions sent to one resident vision-language model V. For repeated queries with identical image bytes, the goal is to reduce the time spent before decoding while preserving

$$\text{CachedVLM}(V, I, q_i) = \text{NativeVLM}(V, I, q_i)$$

under the same deterministic generation settings. No reuse is permitted across distinct content identities.

4 Method

The implementation has a drafter, a verifier, and an escape hatch to native generation. The escape hatch is important: speculative decoding is a conditional optimization, not a replacement decoder.

4.1 Drafting from Nearby Text

The drafter indexes token n-grams from a corpus C. At generation time it looks at the current prefix $p = x \| \hat{y}$, searches for suffixes of p that occur in C, and proposes the tokens that followed the best matching span. Longer suffix matches are preferred, with draft length as a secondary signal. Only explicit context is indexed in the reported adaptive mode. An earlier implementation indexed the concatenation of prompt and context; because benchmark prompts already displayed the source passage, that design could draft from the prompt wrapper and blur the boundary between available context and generated history.

Before entering the verifier loop, MachBoost asks whether the generation boundary has a context candidate. If not, the default profile calls native `mlx-lm` generation immediately. An opt-in re-entry profile generates a bounded number of target tokens, checks the context index again, and either enters block verification or resumes native generation from the live cache. Re-entry can recover copied answers whose first token begins a span in retrieved text. It also changes the cache trajectory, so version 0.2.0 disables it by default.

4.2 Verification Against the Target Model

Given a prefix p and draft $d = (d_1, \ldots, d_k)$, the verifier checks how many draft tokens equal the target model's greedy continuation. A verifier-capable runtime scores a block in one target call and compares target argmax tokens at the relevant positions. If the first a draft tokens are accepted, they are committed. On a mismatch, the target's residual token is emitted. When the whole block is accepted, the verifier also returns the next target token already present in the logits. The following round scores that pending token together with the next draft, avoiding a separate target call between accepted blocks.

Under deterministic target arithmetic, the intended invariant is:

$$\mathrm{Boost}(M, x, C, T) = \mathrm{Greedy}(M, x, T)$$

for a maximum generation length T. Batched and scalar floating-point paths can still choose different argmax tokens at a near tie, so this is tested rather than assumed. Every benchmark row records an `output_match` flag from the native and accelerated token IDs.

4.3 Keeping the Cache Path Stable

Verification writes candidate entries to the MLX prompt cache. Accepted entries remain useful; rejected entries must not survive. MachBoost trims the rejected suffix in place when the cache type permits it. If safe trimming is unavailable, the service reconstructs the accepted prefix. Prompt initialization follows `mlx-lm`'s split prefill trajectory, processing the final prompt token separately. Once no further context candidate exists, the service hands the live verifier cache to `mlx_lm.generate_step`; the remaining response therefore uses the runtime's optimized asynchronous decoder instead of a Python-level `argmax`/synchronize loop.

4.4 Two-Level Reuse for an Unchanged Image

The visual adapter computes a SHA-256 identity from the image contents. Local file metadata memoizes hashing but does not define identity; base64 payloads, data URLs, and in-memory images are hashed directly. The first bounded LRU maps this identity to the projected output of the vision tower,

$$h(I) \longmapsto E_V(I).$$

On a miss, Qwen2.5-VL's vision tower processes the pixel tensor and image grid. On a hit, the stored features enter the model through its `cached_image_features` argument. This Qwen-specific bridge matters because the tested architecture accepts cached features but does not expose the generic `encode_image` method used by MLX-VLM's default feature-cache path.

Feature reuse still leaves roughly 1,000 visual positions for the language model to prefill. MachBoost therefore maintains a second bounded map from $h(I)$ to an MLX-VLM prompt-cache state. When a new question shares a partial token prefix with the prior request for that image, MLX-VLM trims the matching prefix and reuses its KV state. The prompt state is never shared between different image identities. An exact full-prompt match is not trimmed by the upstream partial-prefix path; the benchmark retains one such row as a feature-only control.

Both caches belong to one loaded model instance and are cleared together on reset or unload. This design avoids cross-model tensor reuse and makes image replacement an ordinary cache miss. It does not alter sampling, logits, model parameters, or generated-token decoding.

4.5 Deciding When to Use the Drafter

Drafting from local text is sometimes the wrong choice. Open-ended prompts often accept only a few draft tokens, so verification overhead dominates. MachBoost therefore exposes a benchmark step. For a prompt family, it measures:

- token match between baseline and boosted output,
- wall-clock speedup,
- accepted draft tokens divided by generated tokens,
- target-call or forward-call reduction.

The high-level API can also calibrate an accelerator across representative prompts:

```
from machboost import Accelerator, GatePolicy

boost = Accelerator.from_mlx(model, context_paths=["./docs", "./src"])
calibration = boost.calibrate(
    prompts,
    max_tokens=32,
    gate_policy=GatePolicy(min_speedup=1.05,
                           min_acceptance_rate=0.10),
)
text, stats = boost.generate(prompt, max_tokens=128)
```

If the measured workload fails the policy, `generate` uses the native backend path. Calibration and the per-request initial-candidate gate serve different purposes: calibration enables a workload family, while the gate prevents verifier overhead on an individual request with no draft at its boundary.

5 Implementation

MachBoost is implemented as a small Python package with no required runtime dependencies. Optional extras install backend-specific dependencies:

- `machboost[mlx]` for MLX and `mlx-lm`,
- `machboost[hf]` for PyTorch and Transformers,
- `machboost[vision]` for MLX-VLM,
- Ollama HTTP support for benchmarking and capability detection.

The core package defines a `StepService` protocol with `next_token(prefix)` and an optional `verify(prefix, candidate)` hook. A service with only `next_token` can use the wrapper but cannot skip target work. A verifier can accept several draft tokens per target call. The high-level `Accelerator` loads MLX or Hugging Face models, reads strings, files, or directories as context, and exposes `benchmark`, `calibrate`, and `generate`.

Version 0.2.0 added a resident HTTP process. It owns loaded accelerators, serializes requests per model, retains each model indefinitely by default, and supports finite idle lifetimes or explicit unloading. The CLI can start this process automatically, preload a short model alias such as `qwen2.5:3b`, and issue chat or raw completion requests without reloading weights. The server exposes `/api/chat`, `/api/generate`, model lifecycle endpoints, and OpenAI-compatible chat and completion endpoints. Native MLX streaming forwards `mlx-lm`'s already-detokenized text segments; verified multi-token spans use a stateful streaming detokenizer. This avoids the earlier cost of decoding the entire token prefix once per emitted token.

Version 0.3.0 adds `qwen2.5-vl:3b`, image arguments in the CLI and Python client, Ollama-style image payloads, and OpenAI-style image content parts. MLX objects are created and used on one dedicated worker thread because MLX arrays and streams are thread-affine. The worker owns model load, image preparation, cache lookup, generation, and cache release.

The Ollama HTTP adapter remains a wrapper: Ollama's public generation API does not expose the target token IDs, logits, mutable KV cache, and block-verification hook this implementation needs. Native integration would require a runner-level change rather than an HTTP option.

6 Evaluation

6.1 Machine and Model

Measurements were collected on an Apple M1 Max with 10 CPU cores and 32 GB unified memory, running macOS 26.5.2. The environment used Python 3.13.3 and MLX 0.31.2. Text experiments used `mlx-lm` 0.31.3 and `mlx-community/Qwen2.5-3B-Instruct-4bit`. The paired decoding artifacts used MachBoost 0.1.4; the resident serving artifact used version 0.2.0. The visual artifact used MachBoost 0.3.0, MLX-VLM 0.5.0, and `mlx-community/Qwen2.5-VL-3B-Instruct-4bit`. No thermal or performance warning was reported immediately before or after the paired text run.

The benchmark requests 64 greedy tokens for three fixture families: literal Python continuation, retrieval-grounded answering, and open-ended writing. Each family is repeated five times with a fresh nonce under both default and re-entry profiles. Baseline and accelerated order alternate. Wall time includes prompt processing. The baseline calls `mlx-lm`'s native streaming generator; it is not MachBoost's `next_token` loop. The artifacts `results/mlx_native_default_qwen25_3b_20260713.json` and `results/mlx_native_reentry_qwen25_3b_20260713.json` contain prompts, token previews, raw times, package versions, hardware data, and each run's order.

6.2 Native-Baseline Results

Profile/path	Match	Median	Base tok/s	MB tok/s	Accepted
Default/code	100%	1.96×	87.46	167.28	51
Default/RAG	100%	1.04×	89.98	91.61	0
Default/open	100%	1.00×	99.95	98.27	0
Re-entry/code	100%	1.62×	72.54	121.38	51
Re-entry/RAG	100%	1.58×	88.92	140.09	30
Re-entry/open	100%	1.08×	94.25	101.57	0

Table 1: Five fresh-nonce repeats per profile and fixture. Speedup is paired wall time; throughput and accepted-token columns are per-column medians. "MB" denotes MachBoost.

The default code fixture begins at a boundary that occurs in the supplied source. It accepts a median 51 of 64 output tokens and reduces logical target forwards by 76.6%. Its paired speedups are 2.44, 1.15, 2.08, 1.96, and 1.91×. The 1.96× median is close to the requested 2× threshold but does not clear it, and the range shows how noisy short local measurements remain.

The default retrieval answer has no candidate at its generation boundary and takes native fallback. With one-token re-entry and 1-gram lookup, the first target token anchors a context span; the verifier then accepts a median 30 tokens and reaches 1.58×. Open-ended generation accepts no draft tokens. Its apparent 1.08× ratio is timing noise rather than an algorithmic gain.

All 30 reported rows match their paired native token sequences. A separate 15-row exploratory run with a three-token re-entry probe had one RAG mismatch after an accepted span. This is why re-entry remains opt-in. For a user-facing system, the useful statistics are coverage, conditional speedup, and equality rate rather than one aggregate median.

6.3 Resident Serving Results

The resident benchmark preloads the same 3B model, then sends five forced 64-token completions and five short streaming chats. It measures wall time at the server or Python streaming client, including prompt processing and detokenization. Table 2 summarizes the artifact `results/resident_qwen25_3b_20260713.json`.

Workload	Rows	Median TTFT	Median total	End-to-end tok/s
Forced 64-token completion	5	–	0.657 s	97.47
Short streaming chat	5	0.298 s	0.358 s	–

Table 2: Resident-server latency after preloading the model. Chat outputs contain nine tokens in each recorded row; TTFT is measured when the client receives the first non-empty text segment.

The first forced completion takes 0.973 seconds because the Metal execution path still needs initialization after weight loading; the following four take 0.650–0.663 seconds. These rows accept no context draft tokens. The measurement therefore demonstrates load amortization, corrected streaming, and native fallback rather than a speculative-decoding gain. A same-day Ollama probe records a 0.883-second median for the same forced-token shape, a 1.35× wall-time ratio relative to the 0.657-second resident run. The two runtimes use different 4-bit model formats and the requests were not interleaved, so this is a backend-suitability observation rather than exact-weight evidence.

6.4 Repeated-Image Results

The visual benchmark uses a generated 1024 by 768 image containing a project name, status, budget, owner, and labeled colored shape. Four short extraction questions are each repeated three times. Every pair sends one request with both visual caches disabled and one request with both enabled; pair order alternates by repetition. Both paths use the same loaded model, greedy decoding, a 16-token limit, and the same image. Before recording, the harness performs one uncached warm-up and one cached prime. Table 3 summarizes `results/vision_cache_qwen25_3b_20260714.json`.

Metric	Uncached	Accelerated	Ratio
Median wall time	2.861 s	0.158 s	18.08×
Median paired wall time	–	–	17.96×
Median time to first text	2.849 s	0.149 s	19.12×
Effective prompt throughput	370.88 tok/s	12324.47 tok/s	33.23×

Table 3: Twelve uncached and 12 accelerated requests over one unchanged image. Effective prompt throughput divides the full logical prompt length by measured prefill time even when cached tokens are not recomputed.

The median of per-pair wall-time ratios is 17.96×; individual ratios are retained in the artifact. All 12 accelerated outputs exactly equal their paired uncached outputs, and both modes answer every fixture question correctly. Projected features hit in all recorded accelerated rows. Eleven rows reuse a median 1,018-token visual prefix and range from 13.83× to 19.49×. The feature-only control reaches 1.51×.

The distinction between those rows explains most of the result. Skipping the vision tower alone removes useful work, but the language model still processes the visual token span. Reusing its KV prefix reduces the second stage to the changed question suffix. Generated answers are only two to five tokens long, so time to first text dominates total latency. Generation throughput after the first token is not improved by this mechanism.

6.5 Why Earlier Ratios Were Rejected

Earlier repository artifacts reported 3–8× on MLX. Their baseline disabled the prompt cache or performed a synchronized target call for each token. That path ran near 10 tokens/s and was not representative of native `mlx-lm`, which uses cached asynchronous generation. Those artifacts remain available as implementation diagnostics, but they cannot support a production speed claim. Replacing that baseline was also what exposed a regression in MachBoost's serial tail; switching the tail to native generation restored parity for non-overlap prompts.

A one-repeat Hugging Face artifact likewise found MachBoost context lookup competitive with Transformers prompt lookup, but it predates the current paired harness. It is useful for designing a larger comparison, not for the headline result.

6.6 Exploratory Alternatives

We tested several ways to extend acceleration beyond literal context overlap. Table 4 summarizes the local probes. They were not all run under one benchmark protocol, so the numbers are diagnostic rather than comparative.

Path	Local result	Main constraint
DFlash, Qwen3.5 9B	0.66× at 128; 0.96× at 512	M1 lacks the newer accelerator path; draft emulation costs more than accepted work saves.
Qwen2.5 0.5B draft for 3B target	0.38–0.78×	Acceptance of roughly 36–58% did not repay serial draft generation.
Ollama embedded MTP	12.7–24.3 tok/s versus roughly 40 tok/s baseline	The tested draft depths added overhead on this model and machine.
ANE 0.5B draft	51.1 decode tok/s	A one-token ANE draft cannot feed a 3B MLX target near 102 tok/s economically; multi-token heads are needed.
Lossless Q4 compression	1.08–1.15× ideal entropy bound	The quantized weights are already close to high entropy; decoder cost would reduce the gain further.

Table 4: Exploratory paths that did not beat native generation locally. Each result is specific to the tested model, implementation, and M1 Max.

The failures share a simple accounting problem. A draft only helps when the time removed from target decoding exceeds draft generation, verification, synchronization, and cache repair. A smaller model is not automatically a cheap enough draft. Moving it to another processor is not sufficient either if it predicts one token at a time. The strongest published results solve this with trained multi-token prediction, high acceptance, or concurrent heterogeneous execution rather than with process priority.

7 Limitations

The text evidence has 30 headline rows, one model, one quantization, one machine, and controlled 64-token prompts. The resident evidence adds ten requests but does not broaden model or hardware coverage. The accelerated code fixture is intentionally favorable: the desired code is substantially present in context. It demonstrates the mechanism but does not estimate how often real users encounter an eligible boundary. Longer repository sessions, multiple 3B–9B architectures, prefill-heavy prompts, concurrent clients, and energy measurements remain untested under the corrected harness.

The visual evidence is smaller still: one synthetic image, four extraction questions, three repeats, one 3B architecture, short answers, and one Apple Silicon machine. It excludes model load and starts after an explicit prime. The result says nothing about a first image view, changed images, long-form visual reasoning, image preprocessing policy, real datasets, memory pressure under many cached images, or concurrent sessions. Since decode work is unchanged, the ratio will shrink as answer length grows. Audio and video do not yet have MachBoost adapters.

Only greedy decoding is evaluated. Exact output equality under the tested execution paths does not prove equality for sampling, beam search, or every floating-point near tie. Ollama is also not accelerated through its public HTTP API.

Finally, this work introduces neither prompt lookup nor prompt caching. The contribution is an adaptive package path, MLX cache integration, a Qwen visual-feature bridge, content-scoped cache composition, native fallback, and reproducible baseline correction. A stronger research claim needs broader comparisons with LLMA, Prompt Lookup Decoding, PLD+, MLX-VLM's current server, and production prefix-cache systems.

8 Next Experiments

There are two plausible tracks. The first keeps models unchanged: add an online eligibility predictor, extend bounded re-entry beyond the current one-token experiment, and add native verifier hooks for llama.cpp/Ollama. The resident server makes this testable under long sessions; evaluation should report request coverage, cold and warm time to first token, decode throughput, peak memory, concurrency, energy, and exact outputs against each backend's own optimized generator.

The second track accepts model-specific preparation. A multi-token drafter distilled from the target could run on the Apple Neural Engine while target verification runs on the GPU, following the heterogeneous lesson of Mirror-SD. Quantized draft weights and KV state could reduce memory traffic in long contexts. Both ideas exceed the scope of a wrapper and should be treated as separate research prototypes with quality and compatibility tests.

The immediate visual work is replication: repeat the paired protocol on several VLM families and sizes, replace the generated card with public OCR, chart, document, and scene datasets, sweep image resolution and answer length, and measure memory and concurrency. A direct comparison with MLX-VLM's server should separate MachBoost's Qwen bridge and content identity from upstream feature and prefix caching. Video should be treated separately; useful candidates include frame-feature reuse, temporal change detection, and diffusion activation caches, each with task-level quality checks.

9 Conclusion

MachBoost now has two conditional acceleration paths. On a 3B text model, literal context-backed code continuation reaches a 1.96× median over five paired runs; one-token re-entry reaches 1.58× on copied retrieval answers. On a 3B visual model, combining projected-feature reuse with an image-scoped visual KV prefix reaches a 17.96× median paired speedup for short repeated questions over one unchanged image. Reported outputs remain equal to their paired deterministic baselines. Neither result is universal. Text acceleration depends on continuation overlap; visual acceleration depends on image reuse and primarily removes prefill. The package makes those conditions visible in per-request metrics and keeps the ordinary backend path available when they do not hold.

References

[1] Apple Machine Learning Research. Mlx: An array framework for apple silicon. https://github.com/ml-explore/mlx, 2023. Software.

[2] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025.

[3] Nikhil Bhendawade, Kumari Nishu, Arnav Kundu, Chris Bartels, Minsik Cho, and Irina Belousova. Mirror speculative decoding: Breaking the serial barrier in llm inference. Apple Machine Learning Research, 2025.

[4] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

[5] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[6] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934*, 2023.

[7] Hugging Face. Assisted decoding. https://huggingface.co/docs/transformers/en/assisted_decoding, 2026. Accessed July 6, 2026.

[8] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR, 2023.

[9] MLX-VLM contributors. MLX-VLM: Vision language models on apple silicon. https://github.com/Blaizzy/mlx-vlm, 2026. Software, accessed July 14, 2026.

[10] Prabod Rathnayaka, Fabian Waschkowski, and Lukas Wesemann. BaseRT: Best-in-class LLM inference on apple silicon via native metal. *arXiv preprint arXiv:2607.00501*, 2026.

[11] Apoorv Saxena. Prompt lookup decoding. https://github.com/apoorvumang/prompt-lookup-decoding, 2023. GitHub repository.

[12] Shwetha Somasundaram, Anirudh Phukan, and Apoorv Saxena. Pld+: Accelerating llm inference by leveraging language model artifacts. *arXiv preprint arXiv:2412.01447*, 2024.

[13] Rishabh Tiwari, Haocheng Xi, Aditya Tomar, Coleman Hooper, Sehoon Kim, Maxwell Horton, Mahyar Najibi, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. Quantspec: Self-speculative decoding with hierarchical quantized kv cache. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

[14] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics, 2020.

[15] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[16] Nan Yang, Tao Ge, Liang Wang, Binxing Jiao, Daxin Jiang, Linjun Yang, Rangan Majumder, and Furu Wei. Inference with reference: Lossless acceleration of large language models. *arXiv preprint arXiv:2304.04487*, 2023.

[17] Aonan Zhang, Ray Zhang, Yunfei Cheng, Chong Wang, and Yi Wang. Recurrent drafter for fast speculative decoding in large language models. *arXiv preprint arXiv:2403.09919*, 2024.

[18] Xin Zhou, Dingkang Liang, Kaijin Chen, Tianrui Feng, Xiwu Chen, Hongkai Lin, Yikang Ding, Feiyang Tan, Hengshuang Zhao, and Xiang Bai. Less is enough: Training-free video diffusion acceleration via runtime-adaptive caching. *arXiv preprint arXiv:2507.02860*, 2025.